

NLP 大作业一实验报告

小组成员：

- 叶墨涵 523030910171
- 邓建斌 523030910177
- 陈祺 523030910169

所属小组： 31 组

计算平台与实验设置

计算平台： 本次实验在本地服务器上完成，使用 GPU A800 进行加速。

实验设置：

- 基底模型：** Qwen2.5-0.5B
- 数据集：** Alpaca 指令微调数据集
- 训练参数：**
 - Epochs: 3
 - Batch Size: 32
 - Learning Rate: 5e-5
 - Optimizer: AdamW
 - Max Sequence Length: 512
- Tokenizer:**
 - 将 eos_token 设置为 `<|im_end|>` 以支持对话结束符。
 - pad_token 设置为 eos_token 以支持批次处理。

代码完成情况

以下为对于各个 TODO 的完成情况

数据预处理与微调 (finetune.py)

- 数据标记化 (Tokenization):**

实现了 `tokenize_function`，利用 `tokenizer.apply_chat_template` 将原始的指令数据 (Instruction/Output) 转换为模型可接受的对话格式。
- 标签构建 (Label Masking):**

在构建 `labels` 时，我们计算了用户输入 (Instruction) 的长度，并将该部分的标签设置为 `-100`。这确保了模型在计算损失 (Loss) 时，只关注 Assistant 回复部分的生成，而不计算 User 指令部分的 Loss，符合指令微调的标准做法。
- 优化器与损失函数:**

使用了 AdamW 优化器 (Learning Rate=5e-5) 和 CrossEntropyLoss (设置 `ignore_index=-100`) 进行模型训练。
- 训练循环:**
 - 获取模型输出 `logits` 后，进行了错位切片操作：`shift_logits = logits[..., :-1, :]` 和 `shift_labels = labels[..., 1:]`。这是因为在时刻 t 的输入应该预测时刻 $t + 1$ 的标签。
 - 最后将张量展平后计算损失并进行反向传播。

文本生成与解码策略 (inference.py)

- 基础生成框架:**

实现了 `generate` 函数作为推理入口，负责处理输入数据的设备转移、参数配置以及根据参数选择不同的解码策略。

- **贪心解码 (Greedy Search):**
在 `_greedy_sample_step` 中实现了贪心策略，每一步直接选择概率最大的 Token (`argmax`) 作为输出。
- **采样解码 (Sampling):**
实现了包含 Temperature、Top-k 和 Top-p (Nucleus Sampling) 的采样策略。
 - **Temperature:** 调节概率分布的平滑程度。
 - **Top-k:** 仅从概率最高的 k 个 Token 中采样。
 - **Top-p:** 仅从累积概率超过 p 的最小 Token 集合中采样。

附加任务完成情况

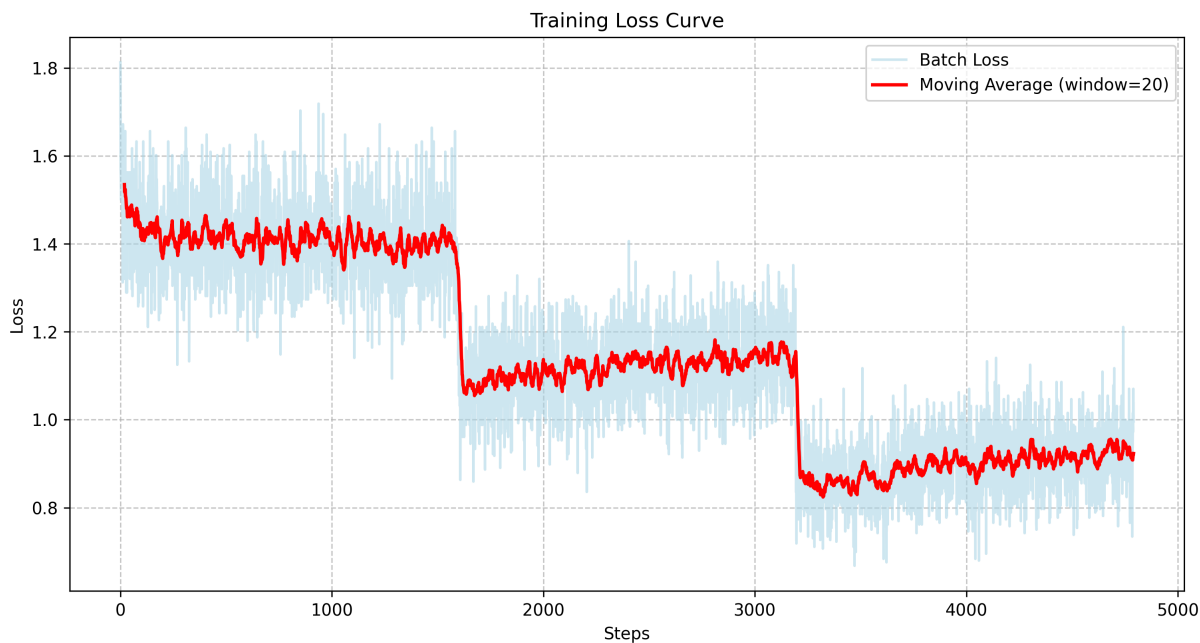
除了基础功能外，我们还完成了以下三个附加任务：

- **批次训练 (Batch Training)**
为了提高训练效率，我们实现了 `collate_fn` 函数。该函数接收一个 batch 的样本列表，提取其中的 `input_ids` 和 `labels`，并使用 `torch.nn.utils.rnn.pad_sequence` 对其进行填充 (Padding)。
 - `input_ids` 填充值为 `tokenizer.pad_token_id`。
 - `labels` 填充值为 `-100`，确保填充部分不参与 Loss 计算。
 - 生成对应的 `attention_mask`，将填充位置设为 0，有效位置设为 1。
- **批次推理 (Batch Inference)**
在 `generate` 函数中，我们首先判断输入 `query` 是否为列表。如果是列表，则直接使用 `tokenizer` 进行批次编码 (`padding=True`)。
在解码循环中，模型一次性接收 `[batch_size, seq_len]` 的输入，并行计算所有样本的下一个 token。我们维护了一个 `finished_sequences` 掩码，用于记录哪些样本已经生成了结束符 (EOS)，从而正确处理不同长度的生成结果。
- **束解码 (Beam Search)**
我们在 `_beam_search_step` 函数中实现了束搜索算法。
 - **初始化:** 将输入序列复制 `num_beams` 份。
 - **迭代生成:** 在每一步计算所有 beam 的下一个 token 的对数概率，并更新累积得分 (Score)。
 - **路径选择:** 从 `batch_size * num_beams * vocab_size` 个候选中选出分数最高的 `num_beams` 个路径。
 - **长度惩罚:** 最终得分除以 `length^penalty`，以平衡生成长度。

训练结果与分析

训练过程

训练时的损失如下图所示



根据损失变化图，可以观察到在 Epoch 1 内，损失先是急剧下降，随后下降趋势变缓。而在 Epoch 2 和 3 中，虽然平均损失均相较上一个 Epoch 大幅下降，但是损失在单个 Epoch 内却进行震荡甚至略微上升。

我们分析，在训练的初期（Epoch 1），模型正在迅速学习微调数据的模式和分布，因此损失迅速下降，而在中后期（Epoch 2, 3），损失已经降到了谷底附近，但是模型的学习率却过大了，就导致了损失产生了震荡或略微上升。可以尝试使用**学习率调度器**，让后期训练时的学习率变小。

生成效果对比

我们使用微调后的模型，对比了不同解码策略的效果：

1. 贪婪解码 (Greedy Search):

- 每次选择概率最大的 Token，生成速度快，但容易陷入局部最优，且在长文本生成中容易出现重复。
- 测试例：

问: 介绍一下上海交通大学。

答: 介绍一下上海交通大学。上海交通大学是1995年9月由复旦、同济、华法、宝山、嘉定... (后续出现大量重复地名)
- 分析: 在上述案例中，贪婪解码在生成中文介绍时陷入了严重的重复循环（Repetition Loop），这是贪婪搜索常见的病态行为。而在英文介绍中，虽然内容基本通顺，但也出现了重复开头的情况。

2. 采样解码 (Sampling):

- 通过调整 temperature , top_p , top_k 引入随机性，生成的文本更具多样性和创造性，能有效避免死循环。
- 测试例：

问: Tell me a joke about computers.

答: Why don't scientists trust computers? Because they make up everything!
- 分析: 采样解码成功生成了多个不同的笑话，且没有出现重复循环的问题。这表明在开放式生成任务（如讲笑话、创意写作）中，采样策略通常优于贪婪策略。

3. 束搜索 (Beam Search):

- 通过探索多条路径，通常能生成逻辑更连贯、语法更准确的句子，尤其在数学推理或长文本生成中表现更好。
- 测试例：

问: What is the sum of the first 100 natural numbers? Please think step by step.

答: The sum of the first n natural numbers is $n(n+1)/2$. Thus, the sum of the first 100 natural numbers is $100(100+1)/2 = 5050$.
- 分析: 在数学推理任务中，束搜索给出了完全正确且逻辑清晰的推导过程。相比之下，贪婪搜索可能因为过早确定错误的路径而导致推理失败。

模型训练结果总结

- 微调前后的变化:** 微调前的基底模型倾向于续写文本，而微调后的模型能够理解 `<|im_end|>`，遵循对话格式，针对用户指令给出回答。
- 模型准确性与幻觉:**
 - 在回答 "What is the capital of China?" 时，贪婪解码虽然答对了 Beijing，但随后开始自问自答列举其他国家首都，甚至出现了 "Sri Jayawardenese Selvan" 这种拼写错误，显示模型在停止生成的控制上仍有不足（可能与 EOS Token 的训练有关）。
 - 在中文生成中出现的严重重复，可能源于模型参数量较小（0.5B），语言模型能力有限，或者训练数据中的中文分布与测试问题不完全匹配。

小组成员分工：

- 叶墨涵：inference.py 中束解码的实现和报告撰写。
- 邓建斌：inference.py 中贪心和采样解码的实现，
- 陈祺：finetune.py 的完成和 README.md 的撰写。